National University of Computer and Emerging Sciences

Assignment No. 2

Generative AI

AI4009

Spring 2026

Submission Instructions:

- Submit the complete .ipynb file, which should be named AI_ASS01_XXF_YYYY, where XX represents the batch and YYYY represents the roll number.
- Submit a Word file containing the GitHub repository link, as well as the Medium and LinkedIn post links
- This is a group assignment (max 2 members allowed).
- Any submission not following the submission instructions will not be evaluated.
- **Submit your own work. Accept your current limitations. Grow from your mistakes.**
- **Plagiarism is strictly prohibited and lead to Straight Zero (^_^).**

Question: Self-Supervised Image Representation Learning using Masked Autoencoders (MAE)

Objective: The objective of this assignment is to design and implement a self-supervised Masked Autoencoder (MAE) system that learns meaningful visual representations by reconstructing images with 75% of input patches masked.

The architecture must follow an asymmetric transformer-based encoder-decoder design, where:

- A large Vision Transformer encoder processes only visible patches.
- A lightweight Vision Transformer decoder reconstructs the masked content.

The entire system must be implemented using base PyTorch layers.

1. Environment Setup

- **Platform:** https://www.kaggle.com/
- **Accelerator:** GPU T4 x2 (Dual GPU)
- **Dataset:** Add the TinyImageNet https://www.kaggle.com/datasets/akash2sharma/tiny-imagenet to your notebook.

2. Model Architecture

➢ **Encoder (Large Vision Transformer)**

Model: ViT-Base (B/16)

Configuration:

- Patch Size: 16 × 16
- Image Size: 224 × 224
- Hidden Dimension: 768
- Transformer Layers: 12

- Attention Heads: 12
- Parameters: ~86 Million

Encoder Requirements:

- Accept only 25% visible patches
- Add positional embeddings
- Do NOT input mask tokens to encoder
- Output latent representations for visible tokens only

➢ Decoder (Lightweight Vision Transformer)

Model: ViT-Small (S/16)

Configuration:

- Patch Size: 16 × 16
- Hidden Dimension: 384
- Transformer Layers: 12
- Attention Heads: 6
- Parameters: ~22 Million

Decoder Requirements:

- Input:
  - Encoder latent tokens
  - Learnable mask tokens for missing patches
- Reconstruct full patch sequence
- Output pixel-level reconstruction

3. Implementation Tasks

Part 1: Patchification & Masking

1. Split 224×224 image into 16×16 patches.
2. Randomly retain exactly **25% patches (49 patches)**.
3. Mask remaining **75% patches (147 patches)**.
4. Maintain proper positional ordering for reconstruction

Part 2: Forward Pass

1. Pass visible patches to encoder.
2. Project encoder output to decoder dimension.
3. Append learnable mask tokens.
4. Apply decoder.
5. Reconstruct pixel patches.
6. Convert patches back to image space.

Part 3: Training Setup

Loss Function

- Mean Squared Error (MSE)
- Compute loss only on masked patches
- Ignore visible patches during loss calculation

Optimizer

- AdamW

Scheduler

- Cosine Learning Rate Scheduler

Required Training Techniques (to fit Kaggle T4×2)

- Mixed Precision (torch.cuda.amp)
- Small batch size (32–64)
- Proper weight decay
- Optional gradient clipping

4. Visualization Module

Create a utility that shows:

- Masked Input (75% removed)
- Model Reconstruction
- Original Ground Truth

Display at least 5 qualitative reconstruction examples.

Deliverables

1. **Jupyter Notebook:**
   - Complete PyTorch implementation.
2. **Training Logs:**
   - Plot reconstruction loss vs epochs.
3. **Quantitative Evaluation:**
   - Minimum 5 reconstructed samples.
   - PSNR Score
   - SSIM Score
4. **App Deployment:**
   - Build a Gradio or Streamlit App that:
     - Accepts image upload
     - Allows masking ratio selection
     - Displays reconstruction in real time